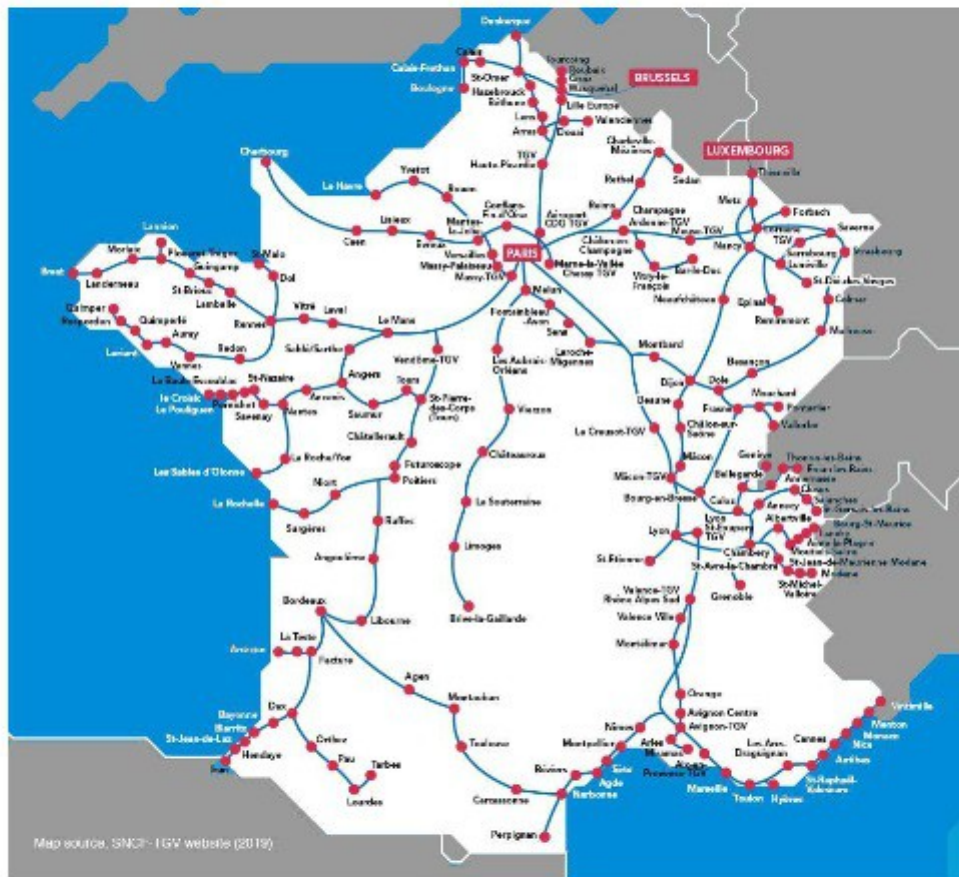


Mon Train



Jean Claude Garcia
Formation : TP Développeur Web et Web Mobile
Année 2026

Sommaire

1.Présentation du projet	page 3
◦ Description	
◦ Spécificités techniques	
2.Analyse et conception	page 3
◦ Étude des besoins fonctionnels et techniques	
◦ Choix technologiques et outils utilisés	
◦ Maquettes et wireframes	
3.Développement	page 4
◦ Différentes étapes du développement	page 4 à 8
◦ Code source et explication	page 9 à 38
◦ Difficultés rencontrées	page 39
4.Tests et validation	page 39
◦ Méthodes de tests	page 39
◦ Résultats des tests	page 40
5.Documentation	page 41
◦ Documentation utilisateur	
◦ Documentation technique	
6.Bilan personnel	page 41
◦ Réflexion	
◦ Point fort et point faible du projet	
7.Annexes	page 42 à 48

1.Présentation du projet

- Description :

Création d'une application web permettant à un utilisateur de trouver un trajet en train pour aller d'une ville à une autre (qui correspond au trajet des trains à grande vitesse en France).

Cette application est disponible via un site web accessible à tous (jeune, personne âgée, personnes en situation de handicap...).

Cette application web full stack permettant de rechercher un trajet en train à grande vitesse entre deux villes françaises, avec gestion des utilisateurs (rôles USER / ADMIN), interface accessible (WCAG 2.1), design responsive, mobile first, multilingue (FR/EN) et multi thèmes de couleur (vert, bleu, rouge, jaune, gris).

- Spécificités techniques :

Conscient de l'importance de la navigation sur mobile aujourd'hui, le site fonctionne de manière optimale sur une variété de dispositifs et de tailles d'écran et est conçu en mobile first.

La sécurité étant un point important, il a été mise en place toutes les bonnes pratiques à ce niveau.

Le site a la possibilité de choisir la langue français ou anglais, ainsi que la palette couleur suivant un dégradé avec comme choix : vert, bleu, rouge, jaune, gris.

Afin de faciliter leur développement, les variables de Bootstrap ont été utilisées .

Cette API permet de récupérer les données dynamiquement et d'interroger une base de données MySQL.

- Authentification (inscription/connexion) avec rôles USER / ADMIN.
- Recherche d'un trajet entre deux villes (ville de départ, ville d'arrivée) sur les lignes de train à grande vitesse.
- Visualisation des lignes de train et de leurs arrêts.
- Interface multilingue (français / anglais).
- Thèmes de couleur sélectionnables (vert, bleu, rouge, jaune, gris).
- Respect des bonnes pratiques d'accessibilité (WCAG 2.1 niveau AA) et W3C.
- Tableau de bord administrateur pour gérer les données (villes, lignes, arrêts, horaires, utilisateurs).

2.Analyse et conception

- Étude des besoins fonctionnels et techniques

Le besoin fonctionnel est relativement simple:

- Créer une application qui s'appuie sur la base de données
- Mettre en place un système d'authentification
- Le MCD du projet au format JPG, PNG ou PDF,

Mon train

- Le MLD au format textuel,
- Alimenter la base de données
- Réaliser une page d'accueil permettant la connexion
- Créer un dépôt GitHub pour versionner votre travail et permettre le partage du code associé

- Choix technologiques et outils utilisés

Pour le Frontend:

- ReactJS
- React Router
- Bootstrap + Sass (mobilefirst, theming via variables)
- i18n (ex : react-i18next)
- Cors

Pour le Backend

- Node.js
- Express
- Sequelize (ORM)
- MySQL ou MariaDB
- JSON Web Tokens (JWT) pour l'authentification
- bcrypt pour le hash mot de passe
- Rate limit
- Helmet (sécurité HTTP)

- Maquettes et wireframes

Voir Annexes

3. Développement

- Différentes étapes du développement

BDD – MCD – Application de recherche de trajets de trains

Les utilisateurs sont caractérisés par :

- Un nom d'utilisateur (username),
- Un numéro de téléphone (phone),
- Un mot de passe (password).

Les caractéristiques des lignes de train sont :

Mon train

- Un nom de la ligne de bus (trainLine),

Les caractéristiques des arrêts de train sont :

- Un nom de l'arrêt de train ou ville (nameCity),

Les caractéristiques des arrêts sont :

- Une ligne de train (trainLineId)
- Une ville (cityId)
- Un numéro d'ordre d'arrêts (stopOrder)

Les caractéristiques des horaires sont :

- Un arrêt de train (lineStopId),
- Un horaire (time),

➤ **UTILISATEUR**

- idUser (PK)
- username (unique)
- phone
- passwordHash
- role (user | admin)

➤ **TRAINLINE**

- idLine (PK)
- trainLine

➤ **CITY**

- idCity (PK)
- nameCity

➤ **LINESTOP (association)**

Représente un arrêt dans une ligne avec son ordre

- idLineStop (PK)
- trainLineId (FK)
- cityId (FK)
- stopOrder

Cardinalités :

Une TRAINLINE possède 1,N LINESTOP

Une CITY apparaît dans 1,N LINESTOP

➤ **SCHEDULE**

Horaire d'un bus à un arrêt précis

- idTime (PK)
- lineStopId (FK)

Mon train

- time

Cardinalités :

Un LINESSTOP possède 0,N SCHEDULE

Un SCHEDULE appartient à 1,1 LINESSTOP

Relations (cardinalités)

UTILISATEUR

→ aucune relation métier avec les autres tables (auth seulement)

TRAINLINE (1,N) — contient — LINESSTOP (1,1)

CITY (1,N) — est desservie par — LINESSTOP (1,1)

LINE_STOP (1,1) — possède — SCHEDULE (0,N)

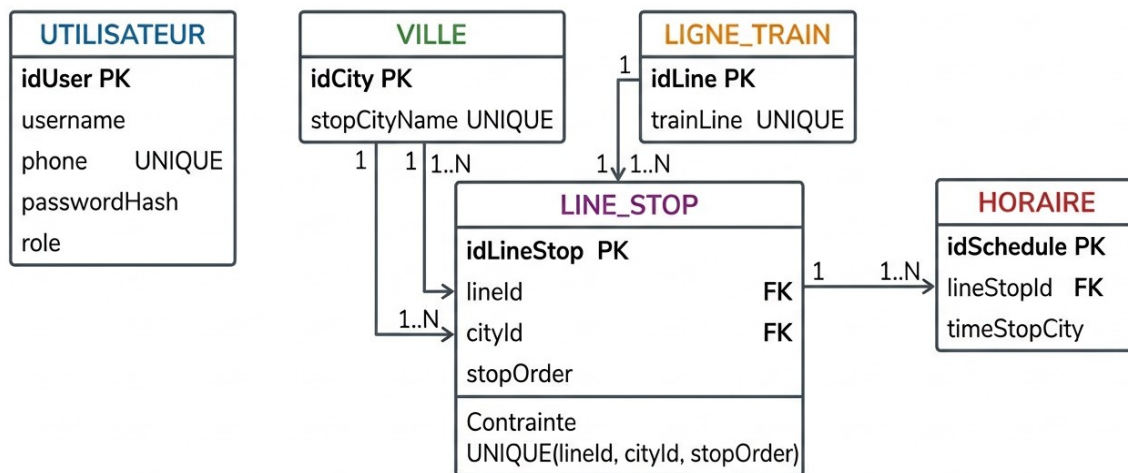
Relations :

TRAIN_LINE (1,N) — LINE_STOP — (N,1) CITY

LINE_STOP (1,1) — (0,N) SCHEDULE

BDD – MLD (Modèle Logique de Données – textuel)

Schéma MLD : Application de Recherche de Trajets en Train



BDD – Création de la Base de données

```
CREATE DATABASE train_app CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
USE train_app;
```

```
-- =====
-- UTILISATEURS
```

```
-- =====
CREATE TABLE users (
  idUser INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  phone VARCHAR(20) NOT NULL,
  passwordhash VARCHAR(255) NOT NULL,
  role ENUM('user','admin') DEFAULT 'user',
);
```

```
-- =====
-- LIGNES DE TRAIN
```

```
-- =====
CREATE TABLE trainLines (
  idLine INT AUTO_INCREMENT PRIMARY KEY,
  nameLine VARCHAR(100) NOT NULL,
);
```

```
-- =====
-- VILLES / ARRETS
```

```
-- =====
CREATE TABLE city (
  idCity INT AUTO_INCREMENT PRIMARY KEY,
  nameCity VARCHAR(100) NOT NULL,
);
```

```
-- =====
-- ASSOCIATION LIGNE ↔ ARRET
```

```
-- =====
CREATE TABLE lineStops (
  idLineStop INT AUTO_INCREMENT PRIMARY KEY,
  trainLineId INT NOT NULL,
  cityId INT NOT NULL,
  stopOrder INT NOT NULL,
```

```
CONSTRAINT fk_trainLine
  FOREIGN KEY (trainLineId) REFERENCES trainLines(idLine)
  ON DELETE CASCADE,
```

```
CONSTRAINT fk_city
  FOREIGN KEY (cityId) REFERENCES city(idCity)
```

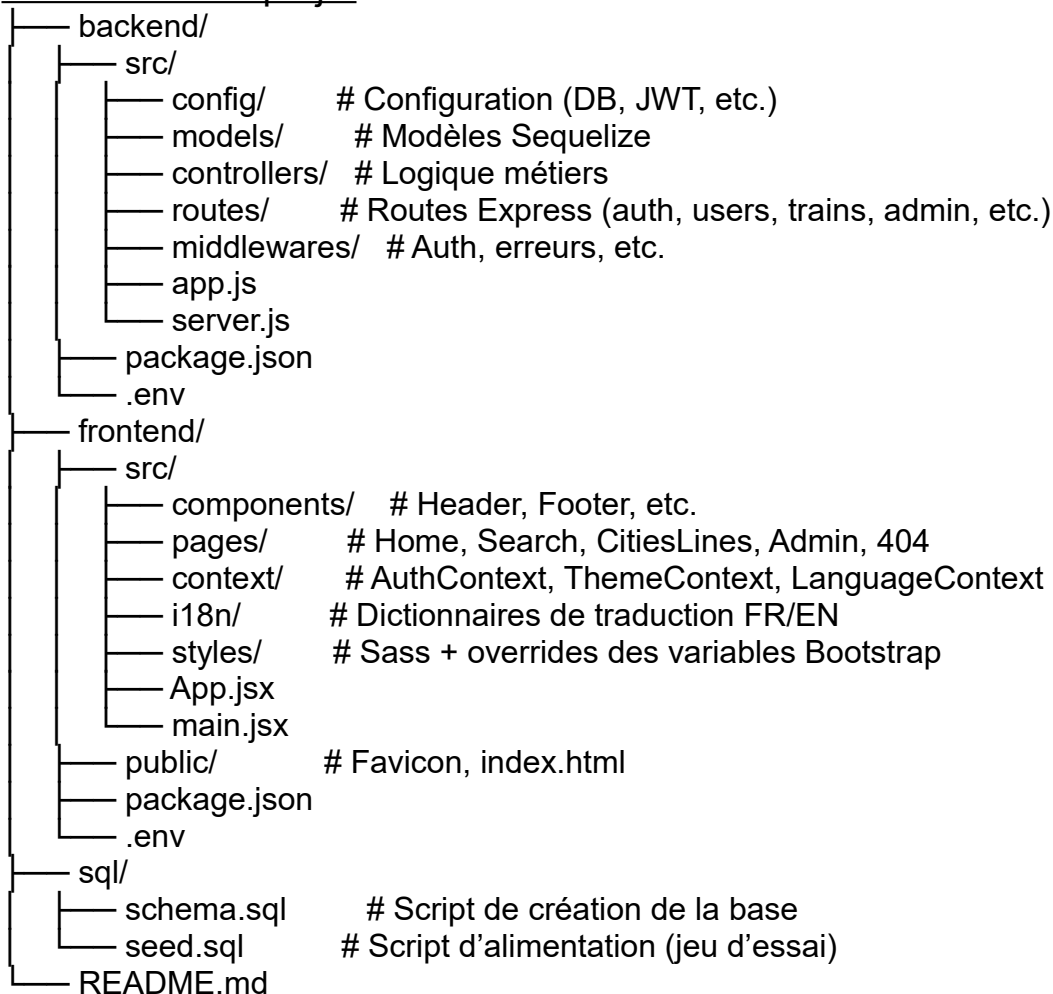
Mon train

```
ON DELETE CASCADE
);

-- =====
-- HORAIRES
-- =====
CREATE TABLE schedules (
  idSchedule INT AUTO_INCREMENT PRIMARY KEY,
  lineStopId INT NOT NULL,
  time VARCHAR(5) NOT NULL,

  CONSTRAINT fk_lineStop
  FOREIGN KEY (lineStopId) REFERENCES lineStops(idLineStop)
  ON DELETE CASCADE
);
```

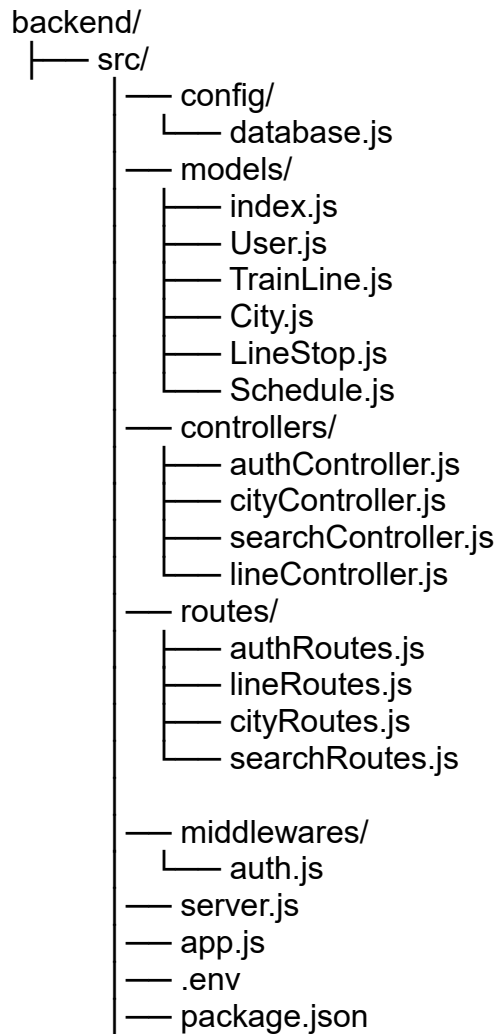
Architecture du projet



- Code source et explication

Backend

Arborescence backend



Installation des dépendances

```
npm init -y
npm install express sequelize mysql2 bcrypt jsonwebtoken dotenv cors helmet express-
rate-limit
npm install nodemon --save-dev
```

app.js

Mon train

```
const app = require('./app');
require('dotenv').config();

const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`Serveur backend démarré sur le port ${PORT}`);
});
```

server.js

```
const express = require('express');
const cors = require('cors');
const dotenv = require('dotenv');
const { sequelize } = require('./models');

dotenv.config();

const app = express();

// CORS
const corsOptions = {
  origin: process.env.CORS_ORIGIN || 'http://localhost:3000',
  credentials: true,
};
app.use(cors(corsOptions));

// Body parser
app.use(express.json());

// Routes
const authRoutes = require('./routes/authRoutes');
const cityRoutes = require('./routes/cityRoutes');
const lineRoutes = require('./routes/lineRoutes');
const searchRoutes = require('./routes/searchRoutes');

app.get('/', (req, res) => {
  res.json({ message: 'API Train Routes OK' });
});

app.use('/auth', authRoutes);
app.use('/cities', cityRoutes);
app.use('/lines', lineRoutes);
app.use('/search', searchRoutes);

// Middleware 404 API
app.use((req, res) => {
```

Mon train

```
res.status(404).json({ message: 'Route introuvable' });
});

// Synchronisation DB
async function initDatabase() {
  try {
    await sequelize.authenticate();
    console.log('Connexion à la base de données réussie.');
```

await sequelize.sync({ alter: false });

```
  } catch (err) {
    console.error('Erreur de connexion à la base de données:', err);
  }
}

initDatabase();

module.exports = app;
```

Mise en place de la sécurité middleware/auth.js

```
const jwt = require('jsonwebtoken');
require('dotenv').config();

function auth(requiredRole = null) {
  return (req, res, next) => {
    const authHeader = req.headers.authorization || '';
    const token = authHeader.startsWith('Bearer ')
      ? authHeader.substring(7)
      : null;

    if (!token) {
      return res.status(401).json({ message: 'Token manquant' });
    }

    try {
      const decoded = jwt.verify(token, process.env.JWT_SECRET);
      req.user = decoded;

      if (requiredRole && decoded.role !== requiredRole) {
        return res.status(403).json({ message: 'Accès refusé' });
      }

      next();
    } catch (err) {
      return res.status(401).json({ message: 'Token invalide' });
    }
  }
}
```

Mon train

```
};  
}
```

```
module.exports = auth;
```

controllers/authController.js

```
const bcrypt = require('bcryptjs');  
const jwt = require('jsonwebtoken');  
const { User } = require('../models');  
require('dotenv').config();
```

```
const SALT_ROUNDS = 10;
```

```
exports.register = async (req, res) => {  
  try {  
    const { username, phone, password } = req.body;  
  
    if (!username || !phone || !password) {  
      return res.status(400).json({ message: 'Champs obligatoires manquants' });  
    }  
  
    if (password.length < 8) {  
      return res  
        .status(400)  
        .json({ message: 'Le mot de passe doit contenir au moins 8 caractères' });  
    }  
  
    const existing = await User.findOne({ where: { phone } });  
    if (existing) {  
      return res.status(409).json({ message: 'Téléphone déjà utilisé' });  
    }  
  
    const passwordHash = await bcrypt.hash(password, SALT_ROUNDS);  
  
    const user = await User.create({  
      username,  
      phone,  
      passwordHash,  
      role: 'USER',  
    });  
  
    return res.status(201).json({  
      idUser: user.idUser,  
      username: user.username,  
      phone: user.phone,  
      role: user.role,  
    });  
  }  
}
```

Mon train

```
});
} catch (err) {
  console.error('Erreur register:', err);
  return res.status(500).json({ message: 'Erreur serveur' });
}
};

exports.login = async (req, res) => {
  try {
    const { phone, password } = req.body;

    if (!phone || !password) {
      return res.status(400).json({ message: 'Champs obligatoires manquants' });
    }

    const user = await User.findOne({ where: { phone } });
    if (!user) {
      return res.status(401).json({ message: 'Identifiants invalides' });
    }

    const isValid = await bcrypt.compare(password, user.passwordHash);
    if (!isValid) {
      return res.status(401).json({ message: 'Identifiants invalides' });
    }

    const token = jwt.sign(
      {
        idUser: user.idUser,
        phone: user.phone,
        role: user.role,
      },
      process.env.JWT_SECRET,
      { expiresIn: process.env.JWT_EXPIRES_IN || '1h' }
    );

    return res.status(200).json({
      token,
      user: {
        idUser: user.idUser,
        username: user.username,
        phone: user.phone,
        role: user.role,
      },
    });
  } catch (err) {
    console.error('Erreur login:', err);
    return res.status(500).json({ message: 'Erreur serveur' });
  }
}
```

Mon train

```
};

exports.getMe = async (req, res) => {
  try {
    const user = await User.findByPk(req.user.idUser, {
      attributes: ['idUser', 'username', 'phone', 'role'],
    });

    if (!user) {
      return res.status(404).json({ message: 'Utilisateur introuvable' });
    }

    return res.status(200).json(user);
  } catch (err) {
    console.error('Erreur getMe:', err);
    return res.status(500).json({ message: 'Erreur serveur' });
  }
};
```

Accès à la Base de données

config/database.js

```
const { Sequelize } = require('sequelize');
const mongoose = require('mongoose');
require('dotenv').config();

const sequelize = new Sequelize(
  process.env.DB_NAME,
  process.env.DB_USER,
  process.env.DB_PASSWORD,
  {
    host: process.env.DB_HOST,
    port: process.env.DB_PORT || 3306,
    dialect: 'mysql',
    logging: false,
  }
);

const MONGODB_URI =
  process.env.MONGODB_URI || 'mongodb://localhost:27017/Api_Train';

const connectMongo = async () => {
  try {
    await mongoose.connect(MONGODB_URI, {
      useNewUrlParser: true,
      useUnifiedTopology: true,
    });
  }
```

```
    console.log('Connected to MongoDB');
  } catch (err) {
    console.error('MongoDB connection error', err);
    process.exit(1);
  }
};
```

```
module.exports = {
  sequelize,
  connectMongo,
  mongoose,
};
```

Composants métiers

Modèles Sequelize

models/User.js

```
const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');
```

```
const User = sequelize.define(
  'User',
  {
    idUser: {
      type: DataTypes.INTEGER,
      primaryKey: true,
      autoIncrement: true,
    },
    username: {
      type: DataTypes.STRING(100),
      allowNull: false,
    },
    phone: {
      type: DataTypes.STRING(20),
      allowNull: false,
      unique: true,
    },
    passwordHash: {
      type: DataTypes.STRING(255),
      allowNull: false,
    },
    role: {
      type: DataTypes.ENUM('USER', 'ADMIN'),
      allowNull: false,
      defaultValue: 'USER',
    },
  },
  {});
```

Mon train

```
{
  tableName: 'utilisateur',
  timestamps: false,
}
);
```

```
module.exports = User;
```

models/TrainLine.js

```
const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');
```

```
const TrainLine = sequelize.define('TrainLine', {
  name: {
    type: DataTypes.STRING,
    allowNull: false,
  },
});
```

```
module.exports = TrainLine;
```

models/City.js

```
const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');
```

```
const StopCity = sequelize.define(
  'StopCity',
  {
    idCity: {
      type: DataTypes.INTEGER,
      primaryKey: true,
      autoIncrement: true,
    },
    cityName: {
      type: DataTypes.STRING(100),
      allowNull: false,
      unique: true,
    },
  },
  {
    tableName: 'ville',
    timestamps: false,
  }
);
module.exports = StopCity;
```


models/TrainLine.js

```
const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');

const LineStop = sequelize.define(
  'LineStop',
  {
    idLineStop: {
      type: DataTypes.INTEGER,
      primaryKey: true,
      autoIncrement: true,
    },
    trainLineId: {
      type: DataTypes.INTEGER,
      allowNull: false,
    },
    stopCityId: {
      type: DataTypes.INTEGER,
      allowNull: false,
    },
    stopOrder: {
      type: DataTypes.INTEGER,
      allowNull: false,
    },
  },
  {
    tableName: 'line_stop',
    timestamps: false,
  }
);

module.exports = LineStop;
```

models/Schedule.js

```
const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');

const Schedule = sequelize.define(
  'Schedule',
  {
    idTime: {
      type: DataTypes.INTEGER,
      primaryKey: true,
    },
  },
  {
    timestamps: false,
  }
);
```

Mon train

```
    autoIncrement: true,
  },
  lineStopId: {
    type: DataTypes.INTEGER,
    allowNull: false,
  },
  time: {
    type: DataTypes.TIME,
    allowNull: false,
  },
},
{
  tableName: 'horaire',
  timestamps: false,
}
);
```

```
module.exports = Schedule;
```

Relations Sequelizeize models/index.js

```
const sequelize = require('../config/database');
const User = require('./User');
const StopCity = require('./StopCity');
const TrainLine = require('./TrainLine');
const LineStop = require('./LineStop');
const Schedule = require('./Schedule');
```

```
TrainLine.hasMany(LineStop, {
  foreignKey: 'linelId',
  as: 'stops',
});
LineStop.belongsTo(TrainLine, {
  foreignKey: 'linelId',
  as: 'line',
});
```

```
StopCity.hasMany(LineStop, {
  foreignKey: 'cityId',
  as: 'lineStops',
});
LineStop.belongsTo(City, {
  foreignKey: 'cityId',
  as: 'city',
});
```

Mon train

```
LineStop.hasMany(Schedule, {
  foreignKey: 'lineStopId',
  as: 'schedules',
});
Schedule.belongsTo(LineStop, {
  foreignKey: 'lineStopId',
  as: 'lineStop',
});
```

```
module.exports = {
  sequelize,
  User,
  City,
  TrainLine,
  LineStop,
  Schedule,
};
```

Mise en place des contrôleurs

controllers/searchController.js

```
exports.searchRoute = async (req, res) => {
  try {
    const { from, to } = req.query;

    if (!from || !to) {
      return res.status(400).json({ message: 'Paramètres from et to obligatoires' });
    }

    const lines = await TrainLine.findAll({
      include: [
        {
          model: LineStop,
          as: 'stops',
          include: [
            {
              model: City,
              as: 'city',
              attributes: ['idCity', 'stopCityName'],
            },
            {
              model: Schedule,
              as: 'schedules',
              attributes: ['idSchedule', 'timeStopCity'],
            },
          ],
        },
      ],
    });
  }
}
```

Mon train

```
    ],
  });

  const results = [];

  lines.forEach((line) => {
    const stops = line.stops.sort((a, b) => a.stopOrder - b.stopOrder);

    const fromStop = stops.find((s) => s.cityId === Number(from));
    const toStop = stops.find((s) => s.cityId === Number(to));

    if (fromStop && toStop && fromStop.stopOrder < toStop.stopOrder) {
      results.push({
        lineId: line.idLine,
        trainLine: line.trainLine,
        from: {
          idCity: fromStop.city.idCity,
          name: fromStop.city.stopCityName,
          schedules: fromStop.schedules,
        },
        to: {
          idCity: toStop.city.idCity,
          name: toStop.city.stopCityName,
          schedules: toStop.schedules,
        },
      });
    }
  });

  return res.status(200).json(results);
} catch (err) {
  console.error('Erreur searchRoute:', err);
  return res.status(500).json({ message: 'Erreur serveur' });
}
};
```

controllers/cityController.js

```
const { City } = require('../models');

exports.getAllCities = async (req, res) => {
  try {
    const cities = await City.findAll({
      order: [['stopCityName', 'ASC']],
    });
    return res.status(200).json(cities);
  } catch (err) {
    console.error('Erreur getAllCities:', err);
  }
};
```

Mon train

```
    return res.status(500).json({ message: 'Erreur serveur' });
  }
};
```

controllers/lineController.js

```
const { TrainLine, LineStop, City, Schedule } = require('../models');

exports.getAllLinesWithStops = async (req, res) => {
  try {
    const lines = await TrainLine.findAll({
      include: [
        {
          model: LineStop,
          as: 'stops',
          include: [
            {
              model: City,
              as: 'city',
              attributes: ['idCity', 'stopCityName'],
            },
            {
              model: Schedule,
              as: 'schedules',
              attributes: ['idSchedule', 'timeStopCity'],
            },
          ],
        },
      ],
      order: [['stopOrder', 'ASC']],
    },
    {
      order: [['trainLine', 'ASC']],
    });

    return res.status(200).json(lines);
  } catch (err) {
    console.error('Erreur getAllLinesWithStops:', err);
    return res.status(500).json({ message: 'Erreur serveur' });
  }
};
```

Mise en place des routes routes/auth.routes.js

```
const express = require('express');
const router = express.Router();
const authController = require('../controllers/authController');
```

Mon train

```
const auth = require('../middlewares/auth');

router.post('/register', authController.register);
router.post('/login', authController.login);
router.get('/me', auth(), authController.getMe);

module.exports = router;
```

routes/cityRoutes.js

```
const express = require('express');
const router = express.Router();
const cityController = require('../controllers/cityController');
const auth = require('../middlewares/auth');

router.get('/', auth(), cityController.getAllCities);

module.exports = router;
```

routes/lineRoutes.js

```
const express = require('express');
const router = express.Router();
const lineController = require('../controllers/lineController');
const auth = require('../middlewares/auth');

router.get('/', auth(), lineController.getAllLinesWithStops);

module.exports = router;
```

routes/searchRoutes.js

```
const express = require('express');
const router = express.Router();
const lineController = require('../controllers/lineController');
const auth = require('../middlewares/auth');

router.get('/search', auth(), lineController.searchRoute);

module.exports = router;
```

Mise en place de l'application

app.js

```
const app = require('./app');
require('dotenv').config();

const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`Serveur backend démarré sur le port ${PORT}`);
});
```

server.js

```
const express = require('express');
const cors = require('cors');
const dotenv = require('dotenv');
const { sequelize } = require('./models');

dotenv.config();

const app = express();

// CORS
const corsOptions = {
  origin: process.env.CORS_ORIGIN || 'http://localhost:3000',
  credentials: true,
};
app.use(cors(corsOptions));

// Body parser
app.use(express.json());

// Routes
const authRoutes = require('./routes/authRoutes');
const cityRoutes = require('./routes/cityRoutes');
const lineRoutes = require('./routes/lineRoutes');
const searchRoutes = require('./routes/searchRoutes');

app.get('/', (req, res) => {
  res.json({ message: 'API Train Routes OK' });
});

app.use('/auth', authRoutes);
app.use('/cities', cityRoutes);
app.use('/lines', lineRoutes);
app.use('/search', searchRoutes);
```

Mon train

```
// Middleware 404 API
app.use((req, res) => {
  res.status(404).json({ message: 'Route introuvable' });
});

// Synchronisation DB
async function initDatabase() {
  try {
    await sequelize.authenticate();
    console.log('Connexion à la base de données réussie.');
```

await sequelize.sync({ alter: false });

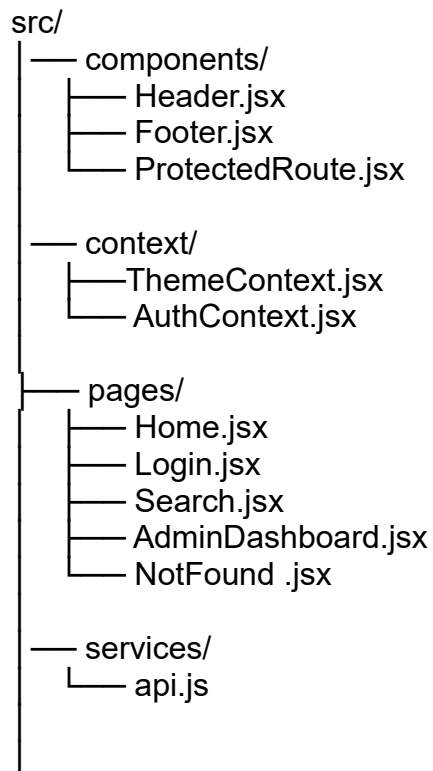
```
  } catch (err) {
    console.error('Erreur de connexion à la base de données:', err);
  }
}

initDatabase();

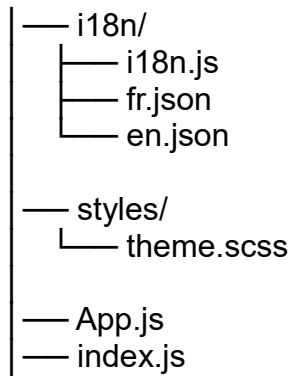
module.exports = app;
```

Frontend

Arborescence Frontend



Mon train



Installation

npx create-react-app frontend

cd frontend

npm install react-router-dom axios bootstrap sass i18next react-i18next

Mise en place du choix de la langue

i18n/i18n.jsx

```
import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';
import fr from './fr.json';
import en from './en.json';

i18n.use(initReactI18next).init({
  resources: { fr: { translation: fr }, en: { translation: en } },
  lng: 'fr',
  fallbackLng: 'en',
  interpolation: { escapeValue: false },
});

export default i18n;
```

fr.json

```
{
  "language": "Langue",
  "theme": "couleur",
  "login": "Se connecter",
  "logout": "Se déconnecter",
  "search": "Rechercher un trajet",
  "from": "Ville de départ",
```

Mon train

```
"to": "Ville d'arrivée",  
"submit": "Lancer la recherche",  
"allRightsReserved": "Tous droits réservés"  
}
```

en.json

```
{  
  "language": "Language",  
  "theme": "Color",  
  "login": "Login",  
  "logout": "Logout",  
  "search": "Search route",  
  "from": "Departure city",  
  "to": "Arrival city",  
  "submit": "Search",  
  "allRightsReserved": "All rights reserved"  
}
```

Mise en place du choix du thème

ThemeContext (choix du thème de la couleur souhaitée)
context/ThemeContext.jsx

```
import { createContext, useState } from 'react';  
  
export const ThemeContext = createContext();  
  
export function ThemeProvider ({ children }) {  
  const [theme, setTheme] = useState('blue');  
  
  const changeTheme = (color) => {  
    document.documentElement.setAttribute('data-theme', color);  
    setTheme(color);  
  };  
  
  return (  
    <ThemeContext.Provider value={{ theme, changeTheme }}>  
      {children}  
    </ThemeContext.Provider>  
  );  
};
```

SASS thème Bootstrap styles/theme.scss

```
:root[data-theme="blue"] {  
  --bs-primary: #0d6efd;  
}  
  
:root[data-theme="green"] {  
  --bs-primary: #198754;  
}  
  
:root[data-theme="red"] {  
  --bs-primary: #dc3545;  
}  
  
:root[data-theme="yellow"] {  
  --bs-primary: #ffc107;  
}  
  
:root[data-theme="grey"] {  
  --bs-primary: #6c757d;  
}
```

Composants statiques components/Footer.jsx

```
import { useTranslation } from "react-i18next";  
  
export default function Footer() {  
  const { t } = useTranslation();  
  
  const year = new Date().getFullYear();  
  
  return (  
    <footer  
      className="bg-light text-center text-lg-start mt-auto py-3 border-top"  
      role="contentinfo"  
    >  
  
      <div className="container text-center">  
  
        <p className="mb-1 fw-bold">  
          Bus App  
        </p>  
      </div>  
    </footer>  
  );  
}
```

Mon train

```
<p className="mb-0 small text-muted">
  © {year} Bus App — {t("allRightsReserved", "Tous droits réservés")}
</p>
```

```
</div>
```

```
</footer>
```

```
);
}
```

Composants dynamiques

Header dynamique

components/Header.jsx

```
import { useContext } from "react";
import { Link } from "react-router-dom";
import { AuthContext } from "../context/AuthContext";
import { ThemeContext } from "../context/ThemeContext";
import { useTranslation } from "react-i18next";

export default function Header() {

  const { user, logout } = useContext(AuthContext);
  const { changeTheme } = useContext(ThemeContext);
  const { t, i18n } = useTranslation();

  return (

    <nav className="navbar navbar-expand-lg navbar-light bg-light border-bottom">

      <div className="container">

        {/* Application name */}

        <Link className="navbar-brand fw-bold" to="/">
          Train App
        </Link>

        {/* Burger */}

        <button
          className="navbar-toggler"
          type="button"
          data-bs-toggle="collapse"
```

Mon train

```
data-bs-target="#navbarContent"
aria-controls="navbarContent"
aria-expanded="false"
aria-label="Toggle navigation"
>
  <span className="navbar-toggler-icon"></span>
</button>

<div className="collapse navbar-collapse" id="navbarContent">

  {/* LEFT MENU */}

  <ul className="navbar-nav me-auto">

    {/* Language */}

    <li className="nav-item dropdown">

      <button
        className="btn nav-link dropdown-toggle"
        data-bs-toggle="dropdown"
      >
        {t("language", "Langue")}
      </button>

      <ul className="dropdown-menu">

        <li>
          <button
            className="dropdown-item"
            onClick={() => i18n.changeLanguage("fr")}
          >
            Français
          </button>
        </li>

        <li>
          <button
            className="dropdown-item"
            onClick={() => i18n.changeLanguage("en")}
          >
            English
          </button>
        </li>

      </ul>
    </li>

  </ul>
</div>
```

Mon train

```
{/* Theme */}

<li className="nav-item dropdown">

  <button
    className="btn nav-link dropdown-toggle"
    data-bs-toggle="dropdown"
  >
    {t("theme", "Couleur")}
  </button>

  <ul className="dropdown-menu">

    <li>
      <button className="dropdown-item" onClick={() => changeTheme("green")}>
        Vert
      </button>
    </li>

    <li>
      <button className="dropdown-item" onClick={() => changeTheme("blue")}>
        Bleu
      </button>
    </li>

    <li>
      <button className="dropdown-item" onClick={() => changeTheme("red")}>
        Rouge
      </button>
    </li>

    <li>
      <button className="dropdown-item" onClick={() => changeTheme("yellow")}>
        Jaune
      </button>
    </li>

    <li>
      <button className="dropdown-item" onClick={() => changeTheme("grey")}>
        Gris
      </button>
    </li>

  </ul>

</li>

</ul>
```

Mon train

```
{/* RIGHT MENU */}

<ul className="navbar-nav ms-auto">

  {!user && (

    <li className="nav-item">

      <Link className="btn btn-primary" to="/login">
        {t("login", "Connexion")}
      </Link>

    </li>

  )}

  {user && (

    <>
      <li className="nav-item">

        <Link className="nav-link" to="/search">
          {t("search", "Recherche")}
        </Link>

      </li>

      <li className="nav-item d-flex align-items-center px-3">

        <span className="small text-muted">
          {user.email}
        </span>

      </li>

      <li className="nav-item">

        <button
          className="btn btn-outline-danger"
          onClick={logout}
        >
          {t("logout", "Déconnexion")}
        </button>

      </li>
    </>
  )}
}
```

```
    </ul>

  </div>

</div>

</nav>
);
}
```

Mise en place de la sécurité

services/api.js

```
import axios from "axios";

const api = axios.create({
  baseURL: "http://localhost:5000/api"
});

api.interceptors.request.use((config) => {

  const token = localStorage.getItem("token");

  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }

  return config;
});

export default api;
```

context/AuthContext.jsx

```
import { createContext, useState } from 'react';
import api from '../services/api';

export const AuthContext = createContext();
export function AuthProvider ({ children }) {

  const [user, setUser] = useState(null);

  const login = async (username, phone, password) => {
    const res = await api.post('/auth/login', { username, phone, password });
```


Mon train

```
localStorage.setItem('token', res.data.token);
const me = await api.get('/auth/me');
setUser(me.data);
};

const logout = () => {
  localStorage.removeItem('token');
  setUser(null);
};

return (
  <AuthContext.Provider value={{ user, login, logout }}>
    {children}
  </AuthContext.Provider>
);
};
```

Mise en place des routes

ProtectedRoute.jsx

```
import { useContext } from "react";
import { Navigate } from "react-router-dom";
import { AuthContext } from "../context/AuthContext";

export default function ProtectedRoute({ children }) {

  const { user } = useContext(AuthContext);

  if (!user) return <Navigate to="/login" />;

  return children;
}
```

Mise en place des pages

Page login

pages/Login.jsx

```
import { useState, useContext } from 'react';
import api from '../services/api';
import { AuthContext } from '../context/AuthContext';

export default function Login() {
  const { login } = useContext(AuthContext);
  const [form, setForm] = useState({ username: "", password: "" });
```

Mon train

```
const handleSubmit = async (e) => {
  e.preventDefault();
  const res = await api.post('/auth/login', form);
  login(res.data.token);
};

return (
  <div className="container mt-5">
    <h1>Connexion</h1>

    <form onSubmit={handleSubmit} className="col-md-4">
      <label className="form-label">Username</label>
      <input
        className="form-control"
        required
        onChange={e => setForm({ ...form, username: e.target.value })}
      />

      <label className="form-label mt-3">Password</label>
      <input
        type="password"
        className="form-control"
        required
        onChange={e => setForm({ ...form, password: e.target.value })}
      />

      <button className="btn btn-primary mt-3">Login</button>
    </form>
  </div>
);
}
```

Page recherche trajet pages/Search.jsx

```
import { useState } from "react";
import api from "../api/axios";

export default function Search() {

  const [from, setFrom] = useState("");
  const [to, setTo] = useState("");
  const [results, setResults] = useState([]);

  const search = async () => {

    const res = await api.get(`/search?from=${from}&to=${to}`);
```

Mon train

```
    setResults(res.data);
  };

  return (
    <div className="container">

      <h2>Recherche trajet</h2>

      <input
        placeholder="Ville départ"
        onChange={(e) => setFrom(e.target.value)}
      />

      <input
        placeholder="Ville arrivée"
        onChange={(e) => setTo(e.target.value)}
      />

      <button onClick={search}>Search</button>

      <ul>
        {results.map((r, i) => (
          <li key={i}>
            {r.departureCity?.name} → {r.arrivalCity?.name}
          </li>
        ))}
      </ul>

    </div>
  );
}
```

Home + Not found pages/Home.jsx

```
export default function Home() {
  return (
    <div className="container mt-5">
      <h1>Bus App</h1>
      <p>1. Connectez-vous</p>
      <p>2. Choisissez une ville de départ</p>
      <p>3. Choisissez une ville d'arrivée</p>
      <p>4. Lancez la recherche</p>
    </div>
  );
}
```

pages/NotFound.jsx

```
export default function NotFound() {
  return (
    <div className="container text-center mt-5">
      <h1>404</h1>
      <p>Page non trouvée</p>
    </div>
  );
}
```

Dashboard admin

pages/AdminDashboard.jsx

```
export default function AdminDashboard() {

  return (
    <div className="container">

      <h2>Admin Dashboard</h2>

      <ul>
        <li><a href="/admin/cities">Cities</a></li>
        <li><a href="/admin/lines">Lines</a></li>
        <li><a href="/admin/schedules">Schedules</a></li>
      </ul>

    </div>
  );
}
```

Mise en place de l'application

App.js

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';

import Header from './components/Header';
import Footer from './components/Footer';
import ProtectedRoute from './components/ProtectedRoute';

import Home from './pages/Home';
import Login from './pages/Login';
import Search from './pages/Search';
```

Mon train

```
import AdminDashboard from "../pages/AdminDashboard";
import NotFound from '../pages/NotFound';
```

```
import { AuthProvider } from "../context/AuthContext";
import { ThemeProvider } from "../context/ThemeContext";
```

```
export default function App() {

  return (

    <AuthProvider>
      <ThemeProvider>

        <BrowserRouter>

          <Header />

          <Routes>

            <Route path="/" element={<Home />} />
            <Route path="/login" element={<Login />} />

            <Route
              path="/search"
              element={
                <ProtectedRoute>
                  <Search />
                </ProtectedRoute>
              }
            />

            <Route
              path="/admin"
              element={
                <ProtectedRoute>
                  <AdminDashboard />
                </ProtectedRoute>
              }
            />

            <Route path="*" element={<NotFound />} />

          </Routes>

          <Footer />

        </BrowserRouter>

      </ThemeProvider>
    </AuthProvider>
  )
}
```

Mon train

```
    </ThemeProvider>
  </AuthProvider>

);
}
```

index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";

import "bootstrap/dist/css/bootstrap.min.css";

import "./i18n/i18n";

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(<App />);
```

.env.example

```
APP_NAME=My_Train_App
API_URL=127.0.0.1
MONGODB_URI=mongodb://localhost:27017/Api_Train

DB_HOST=localhost
DB_PORT=3306
DB_NAME=train_app
DB_USER=root
DB_PASSWORD=jcg3773

FRONT_URL=http://localhost:5000
```

- Difficultés rencontrées

4. Tests et validation

- Méthodes de tests

Tests pour le backend

Jeu de données (seed de test)

-- VILLES

INSERT INTO city (nameCity) VALUES

('Paris'),

('Lyon'),

('Marseille'),

('Lille'),

('Bordeaux'),

('Nantes'),

('Strasbourg');

-- LIGNES

INSERT INTO train_lines (nameLine) VALUES

('TGV Nord-Atlantique'),

('TGV Méditerranée');

-- ASSOCIATIONS (ordre des arrêts)

-- TGV Nord-Atlantique : Lille → Paris → Bordeaux

INSERT INTO lineStops (stopOrder, trainLineId, cityId) VALUES

(1, 2, 4),

(2, 2, 1),

(3, 2, 5);

-- TGV TGV Méditerranée : Paris → Lyon → Marseille

INSERT INTO lineStops (stopOrder, trainLineId, cityId) VALUES

(1, 1, 1),

(2, 1, 2),

(3, 1, 3);

-- HORAIRES

INSERT INTO schedules (time, lineStopId) VALUES

('08h00', 1),

('10h00', 2),

('12h00', 3);

Créer un admin manuellement :

UPDATE users SET role = 'admin' WHERE username = 'admin';

Mon train

Endpoints disponibles

Auth

POST /api/auth/register

POST /api/auth/login

GET /api/auth/me

Recherche

GET /api/search?from=Paris&to=Lyon

Villes (admin pour write)

GET /api/cities

POST /api/cities

PUT /api/cities/:id

DELETE /api/cities/:id

Lignes (admin pour write)

GET /api/train-lines

POST /api/train-lines

DELETE /api/train-lines/:id

Tests pour le frontend

cd frontend

npm install

npm start

Application :

<http://localhost:3000>

- Résultats des tests

5.Documentation

- Documentation utilisateur
- Documentation technique

6.Bilan personnel

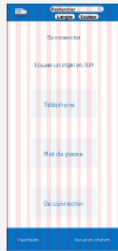
- Réflexion
- Point fort et point faible du projet
 - Projet fonctionnel et structuré.
 - Front-end, API et base de données séparés.
 - Contraintes d'accessibilité et de sécurité intégrées.
 - Projet évolutif pour de futures fonctionnalités.

7. Annexes

Couleur Bleu

Version
FR

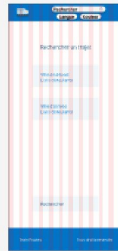
Page Login



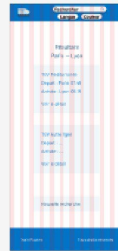
Page Home



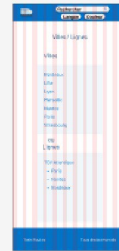
Page Search



Page Resultats



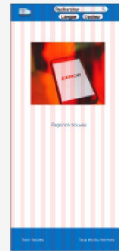
Page Liste ville...



Page Admin da...



Page 404



Version
EN

Page Login



Page Home



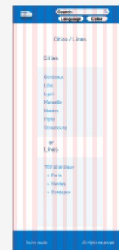
Page Search



Page Resultats



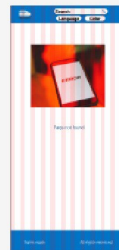
Page Liste ville...



Page Admin da...



Page 404



Mon train

Couleur Vert



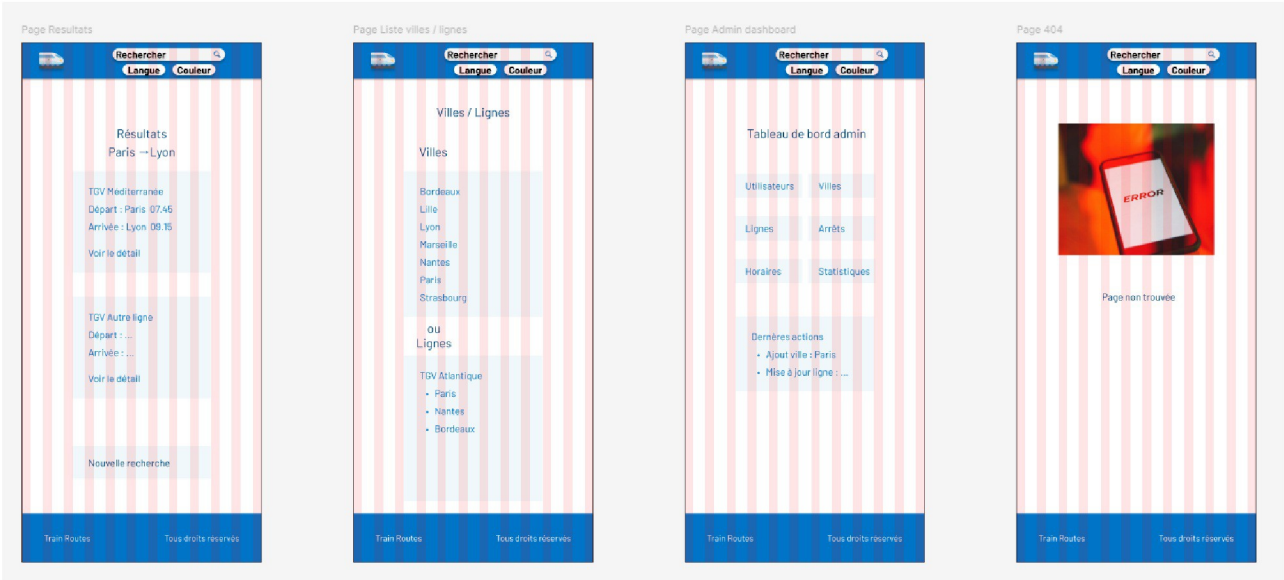
Mon train



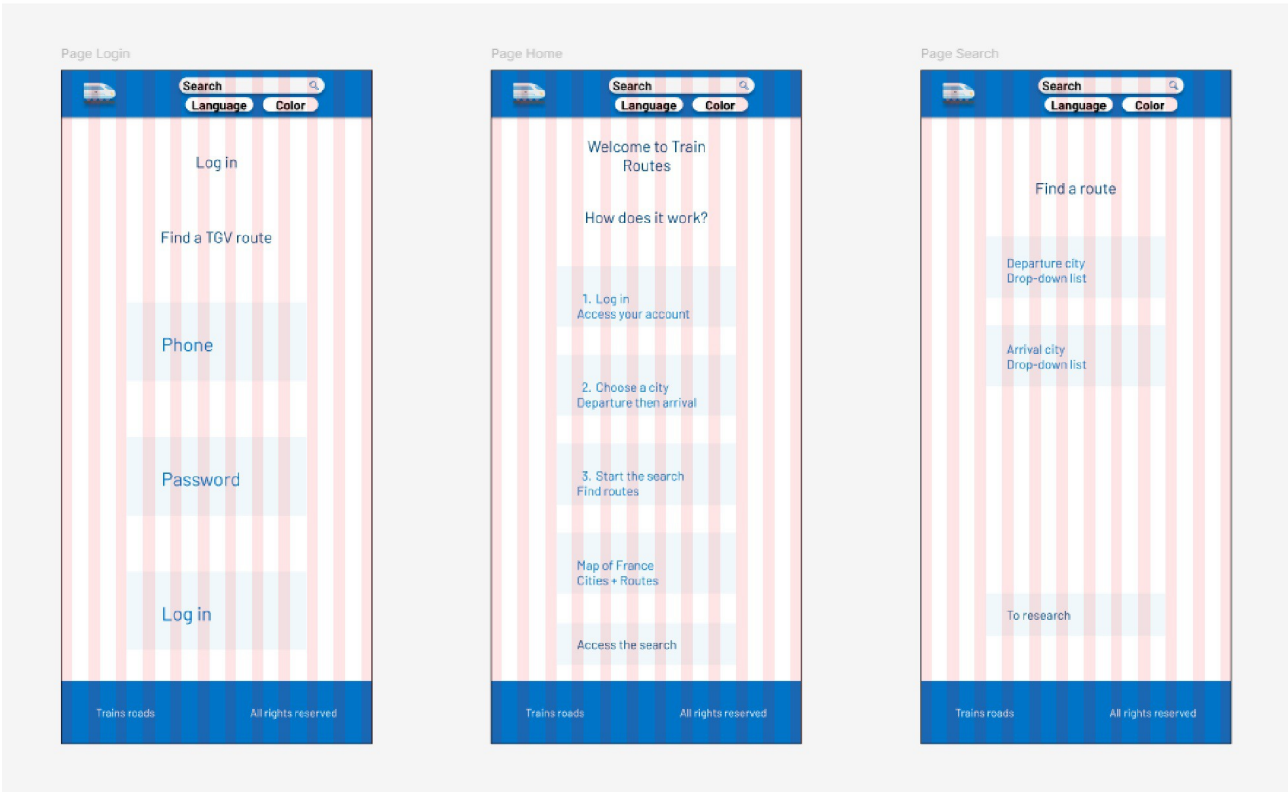
Mon train



Mon train



Mon train



Mon train

